SQL Views – Research & Implementation Task

Objective: Deepen your understanding of **Views** in SQL Server by researching their types and applying them in a realistic, security-focused banking scenario.

Part 1: Research & Documentation

Each team must research and **write a short report** that includes the following:

1. **Types of Views** in SQL Server:
    - Standard View
    - Indexed View
    - Partitioned View (Union View)
2. **For each type**, answer:
    - What is it?
    - Key differences from other view types.
    - Real-life use cases (example from banking, e-commerce, university system…)
    - Limitations and performance considerations.

1. Can We Use DML (INSERT, UPDATE, DELETE) on Views?

- Do some research and explain:
    - **Which types of views** allow DML operations?
    - What are the **restrictions or limitations** when performing DML on a view?
    - Give at least **one real-life example** where updating a view is useful (e.g., HR system, e-commerce orders, etc.)

Tip: Try to test this in SQL Server if possible using a simple view.

2. How Can Views Simplify Complex Queries?

- Explain how a **View** can help simplify **JOIN-heavy queries**.
- Create an example view that joins at least **two of your banking tables**, such as:
    - Customer + Account
    - Account + Transaction
- Show how using the view reduces the need to repeat long queries.

Hint: Think of a scenario where a team needs to access information frequently — like account summaries for call center agents.

Submission Format: PDF or Word document
Best report will be highlighted and used as a template for future batches.

Part 2: Real-Life Implementation Task (Banking System)

You are working for a **Banking System**, and you need to define views to control access and simplify queries for various departments.

Use the following tables (create with dummy data if not already available):

Tables:

```
CREATE TABLE Customer (
   CustomerID INT PRIMARY KEY,
   FullName NVARCHAR(100),
   Email NVARCHAR(100),
   Phone NVARCHAR(15),
   SSN CHAR(9)
);
```

```
CREATE TABLE Account (
    AccountID INT PRIMARY KEY,
    CustomerID INT FOREIGN KEY REFERENCES Customer(CustomerID),
    Balance DECIMAL(10, 2),
    AccountType VARCHAR(50),
    Status VARCHAR(20)
);

CREATE TABLE Transaction (
    TransactionID INT PRIMARY KEY,
    AccountID INT FOREIGN KEY REFERENCES Account(AccountID),
    Amount DECIMAL(10, 2),
    Type VARCHAR(10), -- Deposit, Withdraw
    TransactionDate DATETIME
);

CREATE TABLE Loan (
    LoanID INT PRIMARY KEY,
    CustomerID INT FOREIGN KEY REFERENCES Customer(CustomerID),
    LoanAmount DECIMAL(12, 2),
    LoanType VARCHAR(50),
    Status VARCHAR(20)
);
```

Part 3: View Creation Scenarios

Use **Simple Views** to implement the following:

1. Customer Service View

- Show only customer name, phone, and account status (hide sensitive info like SSN or balance).

2. Finance Department View

- Show account ID, balance, and account type.

3. Loan Officer View

- Show loan details but hide full customer information. Only include CustomerID.

4. Transaction Summary View

- Show only recent transactions (last 30 days) with account ID and amount.